

# Cycle Types in Double Cosets of Young Subgroups via Eulerian Transition Systems

Driven by Dean A. Kulik

September 2026

## Abstract

---

Let  $P$  be a set partition of  $[n]$  with Young subgroup  $K = S_P$ . Double cosets  $K\sigma K$  are indexed by square contingency tables  $T$  with equal row and column sums. We determine the distribution of cycle type inside a double coset, that is, the numbers  $|K\sigma K \cap C_\lambda|$ .

A permutation with table  $T$ , together with a labeling of arcs, corresponds bijectively to a transition system of the Eulerian multidigraph  $G_T$  with adjacency matrix  $T$ , together with a labeling of points. The bijection conjugates the permutation to the successor map on arcs, so cycle type becomes the multiset of closed-trail lengths. This gives  $|K\sigma K \cap C_\lambda| \prod_{i,j} T_{ij}! = TS_\lambda(T) \prod_i d_i!$ , where  $TS_\lambda(T)$  counts transition systems with trail lengths  $\lambda$ .

The BEST theorem then yields  $|K\sigma K \cap C_{(n)}| = t(G_T) \prod_i (d_i - 1)! \prod_i d_i! / \prod_{i,j} T_{ij}!$ . More generally, whenever  $\lambda$  has a unique largest part,  $|K\sigma K \cap C_\lambda|$  is a sum of products of arborescence determinants that is computable in polynomial time for bounded  $n - \lambda_1$ . Involution classes have a closed form that is nonzero exactly when  $T$  is symmetric, so a Young double coset contains an involution if and only if it is self-inverse.

For arbitrary  $\lambda$  we give exact recursions over connected Eulerian sub-tables, characterize the realizable cycle types as closed-trail spectra, and show that the minimum number of cycles equals the number of components of  $G_T$ . We also prove that a nontrivial double coset is never contained in a conjugacy class.

Finally, we classify pairs  $(P, \sigma)$  under conjugation and inversion. We tabulate orbit counts, profile counts and the number of distinct values of  $|K\sigma K \cap C_\lambda|$  through  $n = 9$ , and we measure how much the table

determines the cycle type. All identities are verified exhaustively for  $n \leq 7$  or  $n \leq 8$  by a self-contained companion notebook.

**Keywords:** double cosets, Young subgroups, contingency tables, conjugacy classes, Eulerian digraphs, BEST theorem, transition systems, circuit partition polynomial, involutions.

**MSC 2020:** 05A05, 05A15, 05C30, 05C45, 20B30.

## 1. Introduction

Let  $n \geq 1$ , and let  $P = \{B_1, \dots, B_m\}$  be a set partition of  $[n]$  with block sizes  $d_i = |B_i|$ . Its Young subgroup is  $K = S_P \cong \prod_i S_{d_i}$ . The table of  $\sigma \in S_n$  relative to  $P$  is the  $m \times m$  matrix

$$T(\sigma)_{ij} = |\{x \in B_i : \sigma(x) \in B_j\}|,$$

which records how many points each block sends to each block. Two permutations lie in the same double coset  $K\sigma K$  if and only if they have the same table, and  $|K\sigma K| = \prod_i d_i!^2 / \prod_{i,j} T_{ij}!$  [4, 5, 8]. Under the uniform distribution on  $S_n$ , the table therefore follows the Fisher–Yates distribution. Random permutations drawn from a fixed parabolic double coset have been studied recently, including limit theorems for fixed points [6] and Markov chains on double cosets [5, 7].

This paper determines the cycle-type distribution inside a double coset. For  $\lambda \vdash n$  with conjugacy class  $C_\lambda$  we study

$$M(T, \lambda) = \{\tau \in S_n : T(\tau) = T, \lambda(\tau) = \lambda\} = K\sigma K \cap C_\lambda \quad (T = T(\sigma)).$$

Thus  $|M(T, \lambda)|$  counts the permutations that share with a given permutation both its block-to-block transition counts and its cycle type.

**Main results.** The organizing object is the directed multigraph  $G_T$  on the blocks, with  $T_{ij}$  arcs from  $i$  to  $j$ . Every vertex has equal in-degree and out-degree  $d_i$ .

1. **Transition systems (Theorem 3.1).** Permutations with table  $T$ , decorated by arc labelings, correspond bijectively to transition systems of  $G_T$  decorated by point labelings. The bijection conjugates the permutation to the successor map on arcs. Hence  $|M(T, \lambda)| \prod T_{ij}! = TS_\lambda(T) \prod d_i!$ .
2. **Closed forms (Section 4).** For a single cycle, the BEST theorem gives  $|M(T, (n))|$  as an arborescence count times factorials (Theorem 4.1). For  $\lambda = (L) \cup \nu$  with  $L > \nu_1$ ,  $|M(T, \lambda)|$  is a sum over sub-tables of size  $|\nu|$  (Theorem 4.3). This sum is polynomial-time computable for fixed  $\nu$  (Proposition 4.4) and is explicit for hooks and for classes  $(L, 2^j, 1^k)$ . For involution classes there is a closed form, nonzero exactly when  $T$  is symmetric (Theorem 4.7). Consequently a Young double coset contains an involution if and only if it is self-inverse (Corollary 4.8).
3. **General cycle types (Section 5).** Two exact recursions over connected Eulerian sub-tables compute every  $|M(T, \lambda)|$  without enumerating permutations. The set of realizable cycle types is the set of closed-trail spectra of  $G_T$ , and its minimum number of cycles is the number of connected components of  $G_T$ .

4. **Parity (Theorem 6.2).** A nontrivial double coset contains permutations of both signs. Therefore  $|M| = \min(|K\sigma K|, |C_\lambda|)$  holds only when  $C_\lambda \subseteq K\sigma K$ .
5. **Classification and data (Sections 7 and 8).** The equivalence on pairs  $(P, \sigma)$  generated by simultaneous conjugation and by the relation  $\tau \in M(P, \sigma)$  has classes indexed by profiles  $([T], \lambda)$  (Theorem 7.1). We tabulate these against Burnside orbit counts through  $n = 9$ , and we measure the conditional distribution of cycle type given the table.

**Related work.** The single-cycle formula is the permutation-level, cyclic analogue of Whittle’s formula for the number of sequences with prescribed transition counts [3], and both rest on the BEST theorem [1, 2]. The number of cycles in a double coset is governed by the circuit partition polynomial of  $G_T$ , a normalization of the directed Martin polynomial [9, 10, 11]. Self-inverse double cosets are enumerated in [14]; Theorem 4.7 refines this, for Young subgroups, to the number of involutions in each self-inverse double coset.

**Organization.** Section 2 fixes notation. Section 3 proves the transition-system bijection. Section 4 gives the closed forms, and Section 5 the recursions and the support. Section 6 treats parity, Section 7 the classification of pairs, and Section 8 the conditional distribution of cycle type. Section 9 describes the computations and Section 10 lists open problems.

## 2. Preliminaries

For  $\sigma \in S_n$  write  $\lambda(\sigma) \vdash n$  for its cycle type and  $C_\lambda$  for the conjugacy class, with  $|C_\lambda| = n!/z_\lambda$ . A set partition  $P$  has type  $\mu = (d_1, \dots, d_m)$  when its blocks, listed in decreasing order of size, have sizes  $d_1 \geq \dots \geq d_m$ . Write  $\mathcal{T}_\mu$  for the set of  $m \times m$  nonnegative integer matrices whose row sums and column sums both equal  $(d_1, \dots, d_m)$ .

**Lemma 2.1.** Let  $K = S_P$ . For  $\sigma, \tau \in S_n$ ,  $\tau \in K\sigma K$  if and only if  $T(\tau) = T(\sigma)$ . Every  $T \in \mathcal{T}_\mu$  occurs, and  $|K\sigma K| = \prod_i d_i!^2 / \prod_{i,j} T(\sigma)_{ij}!$ .

*Proof.* For  $k, k' \in K$ ,  $|\{x \in B_i: k\sigma k'(x) \in B_j\}| = |\{y \in B_i: \sigma(y) \in B_j\}|$ , because  $k'$  permutes  $B_i$  and  $k$  preserves  $B_j$ . The converse, and the formula for the size, are standard; see [4, §5] or [8, §1.3]. ◻

Let  $G_T$  be the directed multigraph on vertex set  $[m]$  with  $T_{ij}$  distinguishable arcs from  $i$  to  $j$ , loops included. Its arc set  $A$  has  $|A| = n$ , and each vertex  $v$  has in-degree and out-degree  $d_v$ . For an arc  $a$  write  $\text{tail}(a)$  and  $\text{head}(a)$ .

**Definition 2.2.** A *transition system* of  $G_T$  is a permutation  $\chi$  of  $A$  with  $\text{tail}(\chi(a)) = \text{head}(a)$  for every arc  $a$ . Equivalently, it is a choice at each vertex of a bijection from incoming arcs to outgoing arcs, so there are  $\prod_v d_v!$  transition systems. The cycles of  $\chi$  are closed trails that partition  $A$ . We write  $\lambda(\chi)$  for the multiset of their lengths and  $TS_\lambda(T)$  for the number of transition systems with  $\lambda(\chi) = \lambda$ .

For a connected Eulerian multidigraph  $G$ , let  $t(G)$  be the number of spanning arborescences oriented towards a fixed vertex  $w$ . By the matrix-tree theorem this equals the determinant of the Laplacian  $D_{\text{out}} - A_{\text{adj}}$ , with loops removed from both terms and with the row and column of  $w$  deleted. For Eulerian  $G$  it does not depend on  $w$ . The BEST theorem [1, 2] states that the number of Eulerian circuits of  $G$ , counted as cyclic sequences of distinguishable arcs, is

$$\text{ec}(G) = t(G) \prod_v (d_v - 1)!,$$

where  $d_v$  is the out-degree of  $v$  in  $G$ . We set  $\text{ec}(G) = 0$  when the arcs of  $G$  do not form a single connected piece, and  $\text{ec}$  of the empty graph equal to 1.

### 3. Transition systems and cycle type

**Theorem 3.1.** Fix  $T \in \mathcal{T}_\mu$ . Let  $\Omega(T)$  be the set of pairs  $(\tau, \alpha)$  in which  $T(\tau) = T$  and  $\alpha: [n] \rightarrow A$  is a bijection sending each  $x \in B_i$  with  $\tau(x) \in B_j$  to an arc from  $i$  to  $j$ . Let  $\Psi(T)$  be the set of pairs  $(\chi, \beta)$  in which  $\chi$  is a transition system of  $G_T$  and  $\beta = (\beta_v)_v$  is a family of bijections  $\beta_v: \text{out}(v) \rightarrow B_v$ . Then there is a bijection  $\Omega(T) \rightarrow \Psi(T)$  under which  $\alpha\tau\alpha^{-1} = \chi$ . Consequently  $\lambda(\tau) = \lambda(\chi)$ , and for every  $\lambda \vdash n$

$$|M(T, \lambda)| \cdot \prod_{i,j} T_{ij}! = TS_\lambda(T) \cdot \prod_i d_i!.$$

*Proof.* Given  $(\tau, \alpha)$ , set  $\chi = \alpha\tau\alpha^{-1}$  and let  $\beta_v$  be the restriction of  $\alpha^{-1}$  to  $\text{out}(v)$ . If  $a = \alpha(x)$  runs from  $i$  to  $j$ , then  $\tau(x) \in B_j$ , so  $\chi(a) = \alpha(\tau(x))$  leaves  $j = \text{head}(a)$ . Thus  $\chi$  is a transition system. Each  $\beta_v$  is a bijection onto  $B_v$ , because  $\alpha$  maps  $B_v$  onto  $\text{out}(v)$ .

Conversely, given  $(\chi, \beta)$ , let  $\alpha$  be the inverse of the bijection  $\sqcup_v \beta_v: A \rightarrow [n]$  and set  $\tau = \alpha^{-1}\chi\alpha$ . For  $x \in B_i$  the arc  $\alpha(x)$  leaves  $i$ , say towards  $j$ . Then  $\chi(\alpha(x))$  leaves  $j$ , so  $\tau(x) \in B_j$ . Hence  $T(\tau) = T$ , and  $\alpha$  satisfies the required arc condition. The two constructions are mutually inverse.

Conjugate permutations have the same cycle type, which gives  $\lambda(\tau) = \lambda(\chi)$ . Each  $\tau$  with  $T(\tau) = T$  admits  $\prod_{i,j} T_{ij}!$  labelings  $\alpha$ , and each  $\chi$  admits  $\prod_v d_v!$  families  $\beta$ . Counting both sides of the bijection gives the stated identity. ▀

**Corollary 3.2.**  $\sum_\lambda TS_\lambda(T) = \prod_v d_v!$ . The cycle type of a uniformly random element of the double coset indexed by  $T$  has the same distribution as the multiset of closed-trail lengths of a uniformly random transition system of  $G_T$ .

**Corollary 3.3.** Let  $f_k(G_T)$  be the number of transition systems with exactly  $k$  closed trails, so that  $j(G_T; x) = \sum_k f_k(G_T) x^k$  is the circuit partition polynomial of  $G_T$ . It is related to the directed Martin polynomial by  $j(\vec{G}; x) = x m(\vec{G}; x + 1)$  [9, 10, 11]. Then

$$\sum_{\ell(\lambda)=k} |M(T, \lambda)| = f_k(G_T) \cdot \prod_i d_i! / \prod_{i,j} T_{ij}!.$$

**Example 3.4.** Let  $n = 4$ ,  $P = \{\{1,2\}, \{3,4\}\}$  and  $T = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ . The graph  $G_T$  has a loop at each vertex and one arc in each direction between the two vertices. Its  $2! \cdot 2! = 4$  transition systems have trail lengths  $(2,1,1)$ ,  $(3,1)$ ,  $(3,1)$  and  $(4)$  (Figure 1). With  $\prod d_i! / \prod T_{ij}! = 4$ , Theorem 3.1 gives  $|M(T, (4))| = 4$ ,  $|M(T, (3,1))| = 8$  and  $|M(T, (2,1,1))| = 4$ . These sum to  $|K\sigma K| = 16$ .

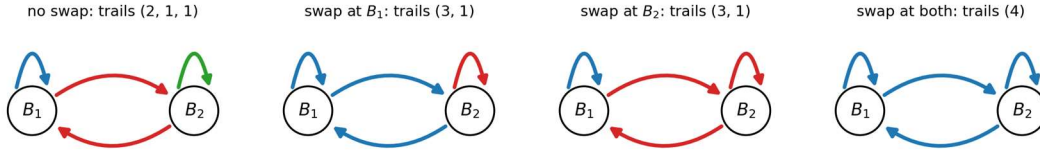


Figure 1. The four transition systems of  $G_T$  for  $T = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ . Each colour is one closed trail.

## 4. Closed forms

**Theorem 4.1.** For every  $T \in \mathcal{T}_\mu$ ,

$$|K\sigma K \cap \mathcal{C}_{(n)}| = |M(T, (n))| = t(G_T) \prod_i (d_i - 1)! \frac{\prod_i d_i!}{\prod_{i,j} T_{ij}!},$$

and  $|M(T, (n))| = 0$  if  $G_T$  is disconnected.

*Proof.* A transition system has a single closed trail exactly when  $\chi$  is one cyclic arrangement of  $A$ , that is, an Eulerian circuit. Hence  $TS_{(n)}(T) = \text{ec}(G_T)$ , and the formula follows from Theorem 3.1.  $\square$

**Remark 4.2.** Whittle's formula [3] counts sequences with prescribed transition counts  $f_{ij}$  as  $\prod_i f_i! / \prod_{i,j} f_{ij}!$  times a cofactor of  $I - D^{-1}F$ , where  $D$  is the diagonal matrix of row sums. That cofactor equals the reduced Laplacian determinant divided by  $\prod_{i \neq w} d_i$ . Theorem 4.1 is the corresponding statement for labeled cyclic objects, transferred to double cosets through Theorem 3.1.

**Theorem 4.3 (dominant cycle).** Let  $\lambda = (L) \cup \nu$ , where  $\nu$  is a partition (possibly empty) with  $L > \nu_1$ . Then

$$TS_\lambda(T) = \sum_R \prod_{i,j} \binom{T_{ij}}{R_{ij}} TS_\nu(R) \text{ec}(G_{T-R}),$$

where  $R$  ranges over integer matrices with  $0 \leq R \leq T$  entrywise, equal row and column sums, and total  $|\nu|$ . Here  $TS_\nu(R)$  counts transition systems of  $G_R$  with trail lengths  $\nu$ .

*Proof.* Since  $L$  exceeds every other part, a transition system with spectrum  $\lambda$  has exactly one closed trail of length  $L$ . The remaining arcs form a balanced sub-multigraph with class counts  $R$ , and they carry a transition system with spectrum  $\nu$ . The binomial factors count the choices of labeled arcs in each class. The long trail is an Eulerian circuit on the complementary arcs.  $\square$

**Proposition 4.4.** Fix a partition  $\nu$  and put  $r = |\nu|$ . For  $T \in \mathcal{T}_\mu$  with  $m$  blocks,  $|M(T, (n-r) \cup \nu)|$  (with  $n-r > \nu_1$ ) can be computed with  $O(m^{2r+3})$  arithmetic operations on integers of  $O(n \log n)$  bits.

*Proof.* There are at most  $\binom{m^2+r-1}{r} = O(m^{2r})$  nonnegative integer  $m \times m$  matrices of total  $r$ , and testing balance costs  $O(m^2)$ . Each  $TS_\nu(R)$  concerns a multigraph with  $r$  arcs, so its cost is bounded in terms of  $r$  alone. Each  $\text{ec}(G_{T-R})$  is a determinant of order less than  $m$ , computed by fraction-free elimination in  $O(m^3)$  operations, times a product of factorials. Every term is at most  $2^n \cdot r! \cdot n^n \cdot n!$ : the binomial product is at

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

[github.com/QuHarmonics/The-Nexus-Harmonic-Reality](https://github.com/QuHarmonics/The-Nexus-Harmonic-Reality) [info@quharmonics.com](mailto:info@quharmonics.com)

most  $2^n$ , the arborescence count is at most  $\prod_v d_v \leq n^n$ , and  $\prod_v (d_v - 1)! \leq n!$ . The intermediate values of fraction-free elimination are minors of the Laplacian and are bounded by Hadamard's inequality. So all integers have  $O(n \log n)$  bits. ▫

**Corollary 4.5 (hooks).** For  $0 \leq k \leq n - 2$ ,

$$TS_{(n-k,1^k)}(T) = \sum_a \prod_v \binom{T_{vv}}{a_v} \text{ec}(G_{T-\text{diag}(a)}),$$

where the sum ranges over integer vectors  $a$  with  $0 \leq a_v \leq T_{vv}$  and  $\sum_v a_v = k$ .

*Proof.* Apply Theorem 4.3 with  $v = (1^k)$ . A closed trail of length one is a loop, and a set of  $k$  loops carries exactly one transition system with spectrum  $(1^k)$ . ▫

**Corollary 4.6.** Let  $L = n - 2j - k \geq 3$ . Then

$$TS_{(L,2^j,1^k)}(T) = \sum_{a,c,b} \prod_v \binom{T_{vv}}{a_v} \binom{T_{vv} - a_v}{2c_v} \frac{(2c_v)!}{2^{c_v} c_v!} \prod_{i < l} \binom{T_{il}}{b_{il}} \binom{T_{li}}{b_{il}} b_{il}! \text{ec}(G_{T-R(a,c,b)}),$$

where  $\sum_v a_v = k$  and  $\sum_v c_v + \sum_{i < l} b_{il} = j$ , and  $R(a, c, b)$  consists of  $a_v + 2c_v$  loops at each vertex  $v$  together with  $b_{il}$  arcs in each direction between  $i$  and  $l$ .

*Proof.* A closed trail of length two is either a pair of loops at one vertex or a pair of opposite arcs between two vertices. The weights count the choices of  $a_v$  single loops, of  $c_v$  loop pairs (matched in  $(2c_v)!/(2^{c_v} c_v!)$  ways), and of  $b_{il}$  opposite-arc pairs (matched in  $b_{il}!$  ways). The rest follows from Theorem 4.3. ▫

**Theorem 4.7 (involutions).** If  $T \neq T^t$  then  $TS_{(2^j,1^k)}(T) = 0$ . If  $T = T^t$  then

$$TS_{(2^j,1^k)}(T) = \prod_{i < l} T_{il}! \cdot [x^k] \prod_v \sum_{\substack{0 \leq a \leq T_{vv} \\ a \equiv T_{vv} \pmod{2}}} \binom{T_{vv}}{a} (T_{vv} - a - 1)!! x^a,$$

with the convention  $(-1)!! = 1$ .

*Proof.* Every trail has length at most two. An arc  $i \rightarrow l$  with  $i \neq l$  must be paired with an arc  $l \rightarrow i$ , which forces  $T_{il} = T_{li}$  and allows  $T_{il}!$  pairings. At a vertex  $v$ , some  $a$  loops close on themselves and the remaining  $T_{vv} - a$  loops are matched in pairs, in  $(T_{vv} - a - 1)!!$  ways. ▫

**Corollary 4.8.** A double coset of a Young subgroup contains an involution if and only if it is self-inverse, that is, if and only if its table is symmetric.

*Proof.* If  $\tau = \tau^{-1}$  then  $T(\tau) = T(\tau)^t$  by Proposition 5.6. Conversely, if  $T$  is symmetric, the term of Theorem 4.7 with  $a_v = T_{vv}$  for all  $v$  equals  $\prod_{i < l} T_{il}! \geq 1$ . ▫

## 5. General cycle types

**Theorem 5.1 (arc-rooted recursion).** Let  $T \neq 0$  and fix an entry  $e$  with  $T_e \geq 1$ . Let  $\mathcal{S}(T, e)$  be the set of integer matrices  $0 \leq S \leq T$  with  $S_e \geq 1$ , equal row and column sums, and connected support. Then

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

$$TS_{\lambda}(T) = \sum_{S \in \mathcal{S}(T, e)} \binom{T_e - 1}{S_e - 1} \prod_{(i,j) \neq e} \binom{T_{ij}}{S_{ij}} \text{ec}(G_S) TS_{\lambda'}(T - S),$$

where  $s$  is the sum of the entries of  $S$ ,  $\lambda'$  is obtained from  $\lambda$  by removing one part equal to  $s$  (the term is zero if  $\lambda$  has no such part), and  $TS_{\emptyset}(0) = 1$ .

*Proof.* Fix one labeled arc  $\hat{e}$  in class  $e$ . In a transition system  $\chi$ , the closed trail through  $\hat{e}$  uses a set of labeled arcs whose class counts form a matrix  $S \in \mathcal{S}(T, e)$ . Given  $S$ , the binomial factors count the choices of those labeled arcs, with  $\hat{e}$  forced. The restriction of  $\chi$  to them is an Eulerian circuit of  $G_S$ , and the restriction to the remaining arcs is an arbitrary transition system of  $G_{T-S}$ . Conversely, every such triple glues to a unique transition system of  $G_T$ . ▫

**Theorem 5.2 (length-rooted recursion).** Let  $L = \lambda_1$ , and let  $m_L$  be the number of parts of  $\lambda$  equal to  $L$ . Then

$$TS_{\lambda}(T) = \frac{1}{m_L} \sum_S \prod_{i,j} \binom{T_{ij}}{S_{ij}} \text{ec}(G_S) TS_{\lambda'}(T - S),$$

where  $S$  ranges over connected balanced matrices  $0 \leq S \leq T$  with total  $L$ , and  $\lambda'$  is obtained from  $\lambda$  by removing one part equal to  $L$ .

*Proof.* A transition system with spectrum  $\lambda$  has exactly  $m_L$  trails of length  $L$ . Selecting one of them and splitting it off is an  $m_L$ -to-one correspondence onto the terms of the sum. ▫

**Corollary 5.3.** The set  $\Lambda(T) = \{\lambda: M(T, \lambda) \neq \emptyset\}$  is the set of length spectra of decompositions of the arcs of  $G_T$  into closed trails. Equivalently, it is the set of spectra obtained from a decomposition into simple directed cycles by merging groups of cycles whose union is connected.

*Proof.* A transition system decomposes  $A$  into closed trails, and following the trails of a closed-trail decomposition defines a transition system. A closed trail splits into simple cycles with connected union. Conversely, a connected union of arc-disjoint cycles is Eulerian and therefore has an Eulerian circuit. ▫

**Proposition 5.4.**  $\min\{\ell(\lambda): \lambda \in \Lambda(T)\}$  equals the number of connected components of  $G_T$ .

*Proof.* A closed trail lies in one component, so every decomposition has at least one trail per component. Each component is a connected Eulerian multidigraph and therefore has an Eulerian circuit, so one trail per component is attained. ▫

**Proposition 5.5.** Let  $T \oplus (1)$  denote  $T$  with an additional block of size one carrying a loop. Then  $|M(T \oplus (1), \lambda \cup (1))| = |M(T, \lambda)|$ . In particular, the set of values of  $|M|$  for  $n - 1$  is contained in the set of values for  $n$ .

*Proof.* The added point must be a fixed point. The restriction to the remaining points is any permutation with table  $T$  and cycle type  $\lambda$ . ▫

**Proposition 5.6.**  $T(\tau^{-1}) = T(\tau)^t$ , and inversion is a bijection  $M(T, \lambda) \rightarrow M(T^t, \lambda)$ .

*Proof.*  $|\{x \in B_i: \tau^{-1}(x) \in B_j\}| = |\{y \in B_j: \tau(y) \in B_i\}| = T_{ji}$ , and inversion preserves cycle type. ▫

With memoization on the residual table, Theorem 5.1 computes all  $|M(T, \lambda)|$  for a table using at most  $\prod_{i,j} (T_{ij} + 1) \leq 2^n$  candidate sub-tables per level. No polynomial bound is known for general  $\lambda$ ; see Problem 2 in Section 10.

## 6. Parity

**Proposition 6.1.** If  $K \neq 1$ , then every double coset  $K\sigma K$  contains both even and odd permutations.

*Proof.* Some block has at least two elements, so  $K$  contains a transposition  $t$ . Both  $\sigma$  and  $t\sigma$  lie in  $K\sigma K$ , and they have opposite signs. ▀

**Theorem 6.2.** Let  $K \neq 1$  and  $\lambda = \lambda(\sigma)$ . Then  $M(P, \sigma) \subsetneq K\sigma K$ , and

$$|M(P, \sigma)| = \min(|K\sigma K|, |C_\lambda|) \Leftrightarrow C_\lambda \subseteq K\sigma K.$$

*Proof.* A conjugacy class has constant sign, so by Proposition 6.1  $K\sigma K \not\subseteq C_\lambda$ . If equality holds with minimum  $|K\sigma K|$ , then  $M = K\sigma K \subseteq C_\lambda$ , which is impossible. Hence equality forces  $M = C_\lambda$ . The converse is clear. ▀

Table 1 counts, over all pairs  $(P, \sigma)$  with  $K \neq 1$ , how often the intersection is strictly smaller than both sets. The number of such pairs is  $(B_n - 1)n!$ , where  $B_n$  is the Bell number.

**Table 1.** Comparison of  $|M(P, \sigma)|$  with  $\min(|K\sigma K|, |C_{\lambda(\sigma)}|)$  over all pairs with  $K \neq 1$ . The last column is the mean of  $\log_2$  of  $\min(|K\sigma K|, |C_{\lambda(\sigma)}|)/|M(P, \sigma)|$ .

| $n$ | pairs with $K \neq 1$ | strictly smaller | equal to the minimum | equal, with double coset the smaller set | mean of $\log_2$ ratio |
|-----|-----------------------|------------------|----------------------|------------------------------------------|------------------------|
| 5   | 6,120                 | 5,730            | 390                  | 0                                        | 1.193                  |
| 6   | 145,440               | 142,929          | 2,511                | 0                                        | 1.685                  |
| 7   | 4,415,040             | 4,396,147        | 18,893               | 0                                        | 2.221                  |

## 7. Classification of pairs

Let  $X = \{(P, \sigma)\}$  be the set of pairs of a set partition of  $[n]$  and a permutation, so that  $|X| = B_n n!$ . The group  $S_n$  acts by  $g \cdot (P, \sigma) = (gP, g\sigma g^{-1})$ . The involution  $\iota: (P, \sigma) \mapsto (P, \sigma^{-1})$  commutes with this action, and together they give an action of  $S_n \times C_2$ . The function  $F(P, \sigma) = |M(P, \sigma)|$  is invariant under both actions, by Lemma 2.1 and Proposition 5.6.

For  $T, T' \in \mathcal{T}_\mu$  write  $T \sim T'$  if  $T' = \pi T \pi^{-1}$  for a permutation  $\pi$  of the blocks that preserves block sizes. Write  $T \sim^t T'$  if, in addition, transposition is allowed. The *profile* of  $(P, \sigma)$  is the pair consisting of the  $\sim$ -class of  $T(\sigma)$  and the cycle type  $\lambda(\sigma)$ . The *transpose-merged profile* uses the  $\sim^t$ -class instead.

**Theorem 7.1.** The equivalence relation on  $X$  generated by the  $S_n$ -action and by  $(P, \sigma) \approx (P, \tau)$  whenever  $\tau \in M(P, \sigma)$  has as its classes exactly the fibers of the profile. Adding  $\iota$ , the classes are exactly the fibers of the transpose-merged profile. In both cases  $F$  is constant on classes, and the number of classes is the sum of  $|\Lambda(T)|$  over one representative  $T$  of each table class.

*Proof.* Conjugation carries  $(P, \sigma)$  to a pair whose table, with blocks listed by size, is a size-preserving relabeling of  $T(\sigma)$ , and it preserves cycle type. The relation  $\approx$  preserves table and cycle type by definition. So the profile is constant on classes.

Conversely, suppose two pairs  $(P, \sigma)$  and  $(P', \sigma')$  have the same profile. Then  $P$  and  $P'$  have the same type. Choose  $g \in S_n$  with  $gP' = P$  that realizes the block relabeling carrying  $T(\sigma')$  to  $T(\sigma)$ . Then  $g\sigma'g^{-1}$  has table  $T(\sigma)$  and cycle type  $\lambda(\sigma)$ , so it lies in  $M(P, \sigma)$ .

With  $\iota$ , tables are additionally transposed (Proposition 5.6). The count follows from Corollary 5.3.  $\square$

By Burnside's lemma,

$$|X/S_n| = \frac{1}{n!} \sum_{g \in S_n} \text{Fix}(g) |Z(g)|, \quad |X/(S_n \times C_2)| = \frac{1}{2n!} \sum_{g \in S_n} \text{Fix}(g) (|Z(g)| + |\{s \in S_n : gs^{-1}g^{-1} = s\}|),$$

where  $\text{Fix}(g)$  is the number of set partitions fixed by  $g$  and  $Z(g)$  is the centralizer of  $g$ . Since profiles are unions of  $S_n$ -orbits, and transpose-merged profiles are unions of  $S_n \times C_2$ -orbits, the corresponding orbit counts bound the profile counts.

**Table 2.** Tables  $(\sum_{\mu} |\mathcal{T}_{\mu}|)$ , table classes, orbit counts, profile counts and the number of distinct values of  $F$ .

| $n$ | tables  | classes $\sim / \sim^t$ | orbits of $S_n$ | orbits of $S_n \times C_2$ | profiles | transpose-merged profiles | values of $F$ |
|-----|---------|-------------------------|-----------------|----------------------------|----------|---------------------------|---------------|
| 1   | 1       | 1 / 1                   | 1               | 1                          | 1        | 1                         | 1             |
| 2   | 3       | 3 / 3                   | 4               | 4                          | 4        | 4                         | 1             |
| 3   | 9       | 6 / 6                   | 10              | 10                         | 10       | 10                        | 3             |
| 4   | 37      | 16 / 16                 | 35              | 35                         | 35       | 35                        | 6             |
| 5   | 177     | 34 / 34                 | 102             | 101                        | 97       | 97                        | 13            |
| 6   | 1,054   | 95 / 92                 | 395             | 372                        | 359      | 344                       | 30            |
| 7   | 7,237   | 236 / 220               | 1,480           | 1,297                      | 1,181    | 1,086                     | 60            |
| 8   | 57,447  | 699 / 613               | 6,590           | 5,267                      | 4,638    | 3,981                     | 129           |
| 9   | 512,664 | 2,069 / 1,674           | 31,087          | 22,321                     | 18,183   | 14,396                    | 293           |

The column of tables is OEIS A321734 [13]. We did not find the orbit, profile or value sequences in the OEIS.

**Remark 7.2.** The profile partition is not invariant under  $\iota$ , because  $T$  and  $T^t$  are often non-isomorphic. It meets 18 of the 372 orbits of  $S_n \times C_2$  at  $n = 6$  in more than one class, and 122 of the 1,297 orbits at  $n = 7$ . The bound by  $|X/(S_n \times C_2)|$  therefore applies to transpose-merged profiles only. Transpose-merged profiles are fewer than orbits of  $S_n \times C_2$  by 4.0%, 7.5%, 16.3%, 24.4% and 35.5% for  $n = 5, \dots, 9$ .

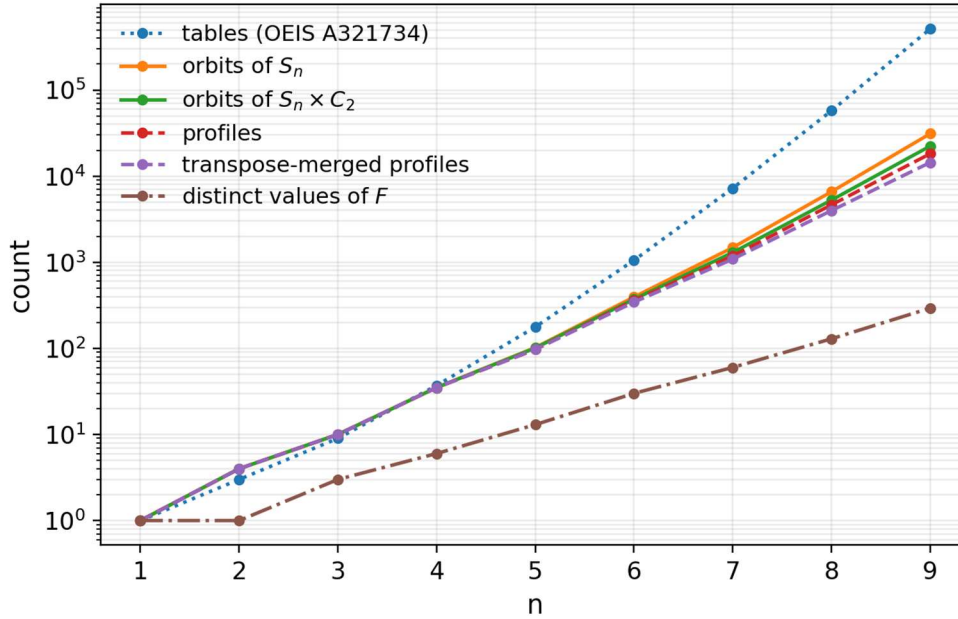


Figure 2. The counts of Table 2 on a logarithmic scale.

## 8. The cycle type given the table

Equip  $X$  with the uniform measure. Write  $H(\lambda)$  for the entropy of the cycle type,  $H(\lambda | T)$  for its conditional entropy given the table,  $I(\lambda; T)$  for their mutual information, and  $H(F)$  and  $H(F | T)$  for the analogous quantities for  $F$ . Let  $P_\lambda$  be the probability that  $\Lambda(T)$  is a singleton, and  $P_F$  the probability that  $|M(T, \cdot)|$  is constant on  $\Lambda(T)$ . All values in Table 3 are exact finite sums over all tables, weighted by the number of set partitions of each type.

Table 3. Information about the cycle type carried by the table (entropies in bits).

| $n$ | $H(\lambda)$ | $H(\lambda   T)$ | $I(\lambda; T)$ | $H(F)$ | $H(F   T)$ | $P_\lambda$ | $P_F$  |
|-----|--------------|------------------|-----------------|--------|------------|-------------|--------|
| 4   | 2.094        | 1.304            | 0.790           | 2.305  | 0.554      | 0.0667      | 0.5000 |
| 5   | 2.557        | 1.718            | 0.839           | 3.051  | 0.828      | 0.0192      | 0.3750 |
| 6   | 3.035        | 2.121            | 0.914           | 3.747  | 1.136      | 0.0049      | 0.2332 |
| 7   | 3.436        | 2.517            | 0.919           | 4.271  | 1.465      | 0.0011      | 0.1349 |
| 8   | 3.843        | 2.903            | 0.940           | 4.862  | 1.791      | 0.0002      | 0.0685 |

Through  $n = 8$  the table carries less than one bit of information about the cycle type, while the conditional entropy  $H(\lambda | T)$  grows with  $n$ . The probability that the table alone determines  $F$  falls from  $1/2$  at  $n = 4$  to  $0.0685$  at  $n = 8$ .

The image of  $F$  is small compared with the number of profiles, but most of its level sets contain profiles with different block types and different numbers of cycles (Table 4). Apart from Proposition 5.5 we know of no bijection relating such profiles, so the size of the image reflects coincidences among the integers of Theorem 3.1 rather than an additional equivalence on pairs.

Table 4. Level sets of  $F$  on transpose-merged profiles.

| $n$ | transpose-merged profiles | values of $F$ | values shared by $\geq 2$ block types | values shared by $\geq 2$ cycle counts | largest level set         |
|-----|---------------------------|---------------|---------------------------------------|----------------------------------------|---------------------------|
| 6   | 344                       | 30            | 21                                    | 22                                     | $F = 2$ (53 profiles)     |
| 7   | 1,086                     | 60            | 40                                    | 43                                     | $F = 4$ (131 profiles)    |
| 8   | 3,981                     | 129           | 82                                    | 80                                     | $F = 12$ (414 profiles)   |
| 9   | 14,396                    | 293           | 172                                   | 177                                    | $F = 24$ (1,567 profiles) |

## 9. Computation

All computations use exact integer arithmetic. Tables are generated directly from their margins, and  $|M(T, \lambda)|$  is computed by Theorem 5.1 with memoization on residual tables. Arborescence counts are computed by fraction-free elimination, and orbit counts by Burnside's lemma. For  $n \leq 7$ , orbits and the classes of Theorem 7.1 were also obtained independently, by union-find over all permutations. Table 5 lists the exhaustive checks. A self-contained Jupyter notebook, using only the Python standard library, reproduces every number in this paper.

Table 5. Exhaustive verification. Ranges beyond  $n = 7$  require the notebook's extended setting.

| Statement                         | Check                                                                 | Range                                                                       | Cases                           |
|-----------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------|---------------------------------|
| Lemma 2.1                         | $K \times K$ -orbits on $S_n$ equal table classes; Fisher–Yates sizes | all block types, $n \leq 8$                                                 | 57,447 double cosets at $n = 8$ |
| Theorem 3.1,<br>Theorem 5.1       | recursion equals enumeration for every $\lambda$                      | all tables, $n \leq 8$                                                      | 57,447 tables at $n = 8$        |
| Theorem 4.1                       | closed form equals enumeration                                        | all tables, $n \leq 8$                                                      | 57,447 tables at $n = 8$        |
| Theorem 4.3                       | formula equals Theorem 5.1                                            | all nonzero $(T, \lambda)$ with $\lambda_1 > \lambda_2$ , $2 \leq n \leq 7$ | 12,038                          |
| Corollary 4.5                     | formula equals Theorem 5.1                                            | all $(T, k)$ , $2 \leq n \leq 7$                                            | 49,532                          |
| Corollary 4.6                     | formula equals Theorem 5.1                                            | all $(T, j, k)$ with $L \geq 3$ , $n \leq 7$                                | 72,248                          |
| Theorem 4.7                       | formula equals Theorem 5.1                                            | all $(T, k)$ , $2 \leq n \leq 7$                                            | 33,830                          |
| Theorem 5.2                       | formula equals Theorem 5.1                                            | all nonzero $(T, \lambda)$ , $2 \leq n \leq 7$                              | 13,706                          |
| Corollary 5.3,<br>Proposition 5.4 | trail spectra equal enumerated support; minimum equals components     | all tables, $n \leq 8$                                                      | 57,447 tables at $n = 8$        |
| Theorem 7.1                       | union-find classes equal profile counts                               | $n \leq 7$                                                                  | Table 2                         |

## 10. Open problems

---

**Problem 1.** Give a formula for the number of profiles (the sum of  $|\Lambda(T)|$  over representatives of table classes) that sums over conjugacy-class data, as Burnside’s lemma does for orbit counts, rather than over table classes.

**Problem 2.** Proposition 4.4 gives polynomial-time evaluation when  $\lambda$  has a unique largest part and  $n - \lambda_1$  is bounded, and Theorem 4.7 covers involutions. Is  $TS_\lambda(T)$  computable in polynomial time for classes with no dominant part whose parts grow with  $n$ , such as  $\lambda = ([n/2], [n/2])$ ?

**Problem 3.** Characterize  $\Lambda(T)$  intrinsically, and determine the complexity of deciding  $\lambda \in \Lambda(T)$ . Partitioning the edges of an undirected graph into triangles is NP-complete [12]; it is not known whether an analogous hardness result holds for closed-trail spectra of Eulerian multidigraphs.

**Problem 4.** Beyond Proposition 5.5, are there bijections between different profiles with equal values of  $F$ , in particular between profiles with different numbers of cycles?

**Problem 5.** Extend the theory to tables with unequal row and column sums. These correspond to injections between blocks of different partitions, and their transition structures contain open trails as well as closed ones.

**Problem 6.** Determine the asymptotic behaviour of  $I(\lambda; T)$  under the uniform measure on  $X$ , and of the ratio of transpose-merged profiles to  $|X/(S_n \times C_2)|$ . That ratio is 0.960 at  $n = 5$  and 0.645 at  $n = 9$ .

## References

---

- [1] T. van Aardenne-Ehrenfest and N. G. de Bruijn, Circuits and trees in oriented linear graphs, *Simon Stevin* 28 (1951), 203–217.
- [2] C. A. B. Smith and W. T. Tutte, On unicursal paths in a network of degree 4, *Amer. Math. Monthly* 48 (1941), 233–237.
- [3] P. Whittle, Some distribution and moment formulae for the Markov chain, *J. Roy. Statist. Soc. Ser. B* 17 (1955), 235–242.
- [4] P. Diaconis and M. Simper, Statistical enumeration of groups by double cosets, *J. Algebra* 607 (2022), 214–246.
- [5] P. Diaconis, A. Ram and M. Simper, Double coset Markov chains, *Forum Math. Sigma* 11 (2023), e2.
- [6] J. E. Paguyo, Fixed points, descents, and inversions in parabolic double cosets of the symmetric group, arXiv:2112.07728.
- [7] M. Simper, Random transpositions on contingency tables, *J. Theoret. Probab.* (2023).
- [8] G. James and A. Kerber, *The Representation Theory of the Symmetric Group*, Addison-Wesley, 1981.

- [9] P. Martin, *Énumérations eulériennes dans les multigraphes et invariants de Tutte–Grothendieck*, Thèse, Grenoble, 1977.
- [10] M. Las Vergnas, Le polynôme de Martin d'un graphe eulérien, *Ann. Discrete Math.* 17 (1983), 397–411.
- [11] J. A. Ellis-Monaghan and C. Merino, Graph polynomials and their applications II: interrelations and interpretations, arXiv:0806.4699.
- [12] I. Holyer, The NP-completeness of some edge-partition problems, *SIAM J. Comput.* 10 (1981), 713–717.
- [13] OEIS Foundation, Sequence A321734, *The On-Line Encyclopedia of Integer Sequences*.
- [14] L. Schwob, On the enumeration of double cosets and self-inverse double cosets, *Adv. in Appl. Math.* (2025); arXiv:2506.04007.

## Appendix A. Table classes for $n = 4$

One representative is listed for each class under  $\sim$ , together with the double-coset size and the nonzero values of  $|M(T, \lambda)|$ .

**Table A1.** The 16 classes of tables for  $n = 4$ .

| type      | representative $T$                   | double coset size | nonzero counts $M(T, \lambda)$                       |
|-----------|--------------------------------------|-------------------|------------------------------------------------------|
| (4)       | (4)                                  | 24                | (4): 6, (3,1): 8, (2,2): 3, (2,1,1): 6, (1,1,1,1): 1 |
| (3,1)     | (3 0; 0 1)                           | 6                 | (3,1): 2, (2,1,1): 3, (1,1,1,1): 1                   |
| (3,1)     | (2 1; 1 0)                           | 18                | (4): 6, (3,1): 6, (2,2): 3, (2,1,1): 3               |
| (2,2)     | (2 0; 0 2)                           | 4                 | (2,2): 1, (2,1,1): 2, (1,1,1,1): 1                   |
| (2,2)     | (1 1; 1 1)                           | 16                | (4): 4, (3,1): 8, (2,1,1): 4                         |
| (2,2)     | (0 2; 2 0)                           | 4                 | (4): 2, (2,2): 2                                     |
| (2,1,1)   | (2 0 0; 0 1 0; 0 0 1)                | 2                 | (2,1,1): 1, (1,1,1,1): 1                             |
| (2,1,1)   | (2 0 0; 0 0 1; 0 1 0)                | 2                 | (2,2): 1, (2,1,1): 1                                 |
| (2,1,1)   | (1 1 0; 1 0 0; 0 0 1)                | 4                 | (3,1): 2, (2,1,1): 2                                 |
| (2,1,1)   | (1 1 0; 0 0 1; 1 0 0)                | 4                 | (4): 2, (3,1): 2                                     |
| (2,1,1)   | (0 1 1; 1 0 0; 1 0 0)                | 4                 | (4): 2, (2,2): 2                                     |
| (1,1,1,1) | (1 0 0 0; 0 1 0 0; 0 0 1 0; 0 0 0 1) | 1                 | (1,1,1,1): 1                                         |
| (1,1,1,1) | (1 0 0 0; 0 1 0 0; 0 0 0 1; 0 0 1 0) | 1                 | (2,1,1): 1                                           |
| (1,1,1,1) | (1 0 0 0; 0 0 1 0; 0 0 0 1; 0 1 0 0) | 1                 | (3,1): 1                                             |
| (1,1,1,1) | (0 1 0 0; 1 0 0 0; 0 0 0 1; 0 0 1 0) | 1                 | (2,2): 1                                             |
| (1,1,1,1) | (0 1 0 0; 0 0 1 0; 0 0 0 1; 1 0 0 0) | 1                 | (4): 1                                               |

## Appendix B. Counts by block type for $n = 7$

Table B1. Tables, table classes and profiles for  $n = 7$  by block type.

| type            | tables | classes $\sim$ | profiles | classes $\sim^t$ | transpose-merged profiles |
|-----------------|--------|----------------|----------|------------------|---------------------------|
| (7)             | 1      | 1              | 15       | 1                | 15                        |
| (6,1)           | 2      | 2              | 25       | 2                | 25                        |
| (5,2)           | 3      | 3              | 37       | 3                | 37                        |
| (5,1,1)         | 7      | 5              | 47       | 5                | 47                        |
| (4,3)           | 4      | 4              | 44       | 4                | 44                        |
| (4,2,1)         | 12     | 12             | 112      | 10               | 93                        |
| (4,1,1,1)       | 34     | 10             | 65       | 10               | 65                        |
| (3,3,1)         | 16     | 10             | 75       | 10               | 75                        |
| (3,2,2)         | 25     | 15             | 114      | 14               | 106                       |
| (3,2,1,1)       | 67     | 39             | 228      | 31               | 183                       |
| (3,1,1,1,1)     | 208    | 19             | 72       | 19               | 72                        |
| (2,2,2,1)       | 120    | 26             | 125      | 24               | 114                       |
| (2,2,1,1,1)     | 378    | 47             | 151      | 44               | 139                       |
| (2,1,1,1,1,1)   | 1,320  | 28             | 56       | 28               | 56                        |
| (1,1,1,1,1,1,1) | 5,040  | 15             | 15       | 15               | 15                        |
| total           | 7,237  | 236            | 1,181    | 220              | 1,086                     |

## Appendix C. The values of $F$ for $n = 7$

There are 60 distinct values: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 15, 16, 18, 20, 21, 24, 25, 28, 30, 32, 36, 40, 45, 48, 50, 54, 60, 64, 70, 72, 84, 90, 96, 100, 105, 108, 120, 140, 144, 160, 168, 210, 216, 240, 260, 280, 288, 300, 324, 360, 420, 432, 480, 504, 540, 630, 720, 840.

# Cycle types in double cosets of Young subgroups via Eulerian transition systems — computational companion

Dean Kulik, QuHarmonics Research Group (ORCID 0009-0003-3128-8828)

This notebook is self-contained. It uses only the Python standard library, reads no files, and reproduces every computed number in the paper. Cells run top to bottom.

With `RUN_LARGE = False` all checks run for  $n \leq 7$  in under a minute. With `RUN_LARGE = True` the enumeration checks extend to  $n = 8$  and the table-only counts to  $n = 9$ , which takes about four minutes in total.

**Conventions.** Points are  $0, \dots, n - 1$  and a permutation is a tuple  $\mathbf{p}$  with  $\mathbf{p}[\mathbf{x}]$  the image of  $\mathbf{x}$ . A block type  $\mu$  lists block sizes in decreasing order, and its blocks are consecutive runs of points. The table of  $\mathbf{p}$  is  $\mathbf{T}[\mathbf{i}][\mathbf{j}] = \#\{x \in B_i : p(x) \in B_j\}$ .

`RUN_LARGE = False`

```
import itertools, math, time
from collections import Counter, defaultdict
from functools import lru_cache
from math import factorial, comb, prod
```

```
N_ENUM = 8 if RUN_LARGE else 7      # largest n for checks that enumerate per
mutations
N_TABLE = 9 if RUN_LARGE else 7      # largest n for counts computed from tabl
es alone
print("enumeration checks up to n =", N_ENUM, "; table-only counts up to n ="
, N_TABLE)
```

enumeration checks up to  $n = 7$  ; table-only counts up to  $n = 7$

## 1. Core definitions

```
def partitions(n, maxpart=None):
    """Integer partitions of n with parts in decreasing order."""
    if maxpart is None:
        maxpart = n
    if n == 0:
        yield ()
        return
    for k in range(min(n, maxpart), 0, -1):
        for rest in partitions(n - k, k):
            yield (k,) + rest
```

```
def cycle_type(p):
    n = len(p); seen = [False] * n; lengths = []
```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828  
 Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)  
[github.com/QuHarmonics/The-Nexus-Harmonic-Reality](https://github.com/QuHarmonics/The-Nexus-Harmonic-Reality) [info@quharmonics.com](mailto:info@quharmonics.com)

```

for i in range(n):
    if not seen[i]:
        L, j = 0, i
        while not seen[j]:
            seen[j] = True; j = p[j]; L += 1
        lengths.append(L)
return tuple(sorted(lengths, reverse=True))

def inverse(p):
    q = [0] * len(p)
    for i, v in enumerate(p):
        q[v] = i
    return tuple(q)

def block_index(mu):
    b = []
    for i, d in enumerate(mu):
        b += [i] * d
    return b

def table_of(p, b, m):
    T = [[0] * m for _ in range(m)]
    for x, y in enumerate(p):
        T[b[x]][b[y]] += 1
    return tuple(map(tuple, T))

def tables(mu):
    """All square nonnegative integer matrices with row sums and column sums
    equal to mu."""
    m = len(mu); out = []
    def fill_row(j, left, colrem, row):
        if j == m - 1:
            if left <= colrem[j]:
                yield row + (left,)
            return
        for v in range(min(left, colrem[j]), -1, -1):
            yield from fill_row(j + 1, left - v, colrem, row + (v,))
    def rec(i, colrem, rows):
        if i == m:
            if all(c == 0 for c in colrem):
                out.append(tuple(rows))
            return
        for r in fill_row(0, mu[i], colrem, ()):
            rec(i + 1, [c - v for c, v in zip(colrem, r)], rows + [r])

```

```

    rec(0, list(mu), [])
    return out

class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x
    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a != b:
            self.parent[a] = b
    def count(self):
        return len({self.find(i) for i in range(len(self.parent))})

def fisher_yates(T):
    """|K sigma K| for the double coset with table T."""
    d = [sum(r) for r in T]
    return prod(factorial(x) for x in d) ** 2 // prod(factorial(v) for r in T
    for v in r)

def n_set_partitions_of_type(mu):
    n = sum(mu); c = Counter(mu)
    return factorial(n) // (prod(factorial(d) for d in mu) * prod(factorial(k)
    ) for k in c.values()))

def relabel(T, p):
    m = len(T)
    return tuple(tuple(T[p[i]][p[j]] for j in range(m)) for i in range(m))

def transpose(T):
    return tuple(zip(*T))

```

## 2. Eulerian circuits (BEST theorem) and transition-system counts

```

def det_int(M):
    """Exact integer determinant (Bareiss elimination)."""
    M = [list(r) for r in M]; n = len(M)
    if n == 0:
        return 1
    sign, prev = 1, 1
    for k in range(n - 1):
        if M[k][k] == 0:

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

[github.com/QuHarmonics/The-Nexus-Harmonic-Reality](https://github.com/QuHarmonics/The-Nexus-Harmonic-Reality) [info@quharmonics.com](mailto:info@quharmonics.com)

```

    for r in range(k + 1, n):
        if M[r][k] != 0:
            M[k], M[r] = M[r], M[k]; sign = -sign
            break
    else:
        return 0
    for i in range(k + 1, n):
        for j in range(k + 1, n):
            M[i][j] = (M[i][j] * M[k][k] - M[i][k] * M[k][j]) // prev
    prev = M[k][k]
    return sign * M[n - 1][n - 1]

def balanced(S):
    m = len(S)
    return all(sum(S[i]) == sum(S[j][i] for j in range(m)) for i in range(m))

def connected(S):
    m = len(S)
    verts = [v for v in range(m) if sum(S[v]) > 0]
    if not verts:
        return True
    adj = {v: set() for v in verts}
    for i in range(m):
        for j in range(m):
            if S[i][j]:
                adj[i].add(j); adj[j].add(i)
    seen, stack = {verts[0]}, [verts[0]]
    while stack:
        x = stack.pop()
        for y in adj[x]:
            if y not in seen:
                seen.add(y); stack.append(y)
    return len(seen) == len(verts)

def eulerian_circuits(S):
    """ec(G_S): Eulerian circuits of the balanced multidigraph with S[i][j] d
    istinguishable arcs i->j,
    counted as cyclic arc sequences (BEST theorem). Returns 0 if the arcs do
    not form one connected piece."""
    m = len(S)
    verts = [v for v in range(m) if sum(S[v]) > 0]
    if not verts:
        return 1
    if not connected(S):

```

```

        return 0
    L = [[(sum(S[i]) - S[i][i]) if i == j else -S[i][j] for j in verts] for i
in verts]
    arborescences = det_int([row[1:] for row in L[1:]])
    return arborescences * prod(factorial(sum(S[v]) - 1) for v in verts)

def subtables(T, forced=None):
    m = len(T); flat = [v for r in T for v in r]
    ranges = [range(1 if k == forced else 0, v + 1) for k, v in enumerate(flat)]
    for S in itertools.product(*ranges):
        yield tuple(tuple(S[i * m:(i + 1) * m]) for i in range(m))

@lru_cache(maxsize=None)
def TS(T):
    """Theorem 5.1: dict lambda -> number of transition systems of G_T whose
closed trails have lengths lambda."""
    m = len(T); flat = [v for r in T for v in r]
    if sum(flat) == 0:
        return {}: 1
    e = next(k for k, v in enumerate(flat) if v)
    out = Counter()
    for S in subtables(T, forced=e):
        if not balanced(S):
            continue
        c = eulerian_circuits(S)
        if c == 0:
            continue
        Sflat = [v for r in S for v in r]
        w = prod(comb(t - 1, s - 1) if k == e else comb(t, s) for k, (t, s) i
n enumerate(zip(flat, Sflat)))
        R = tuple(tuple(T[i][j] - S[i][j] for j in range(m)) for i in range(m
))
        L = sum(Sflat)
        for lam, cnt in TS(R).items():
            out[tuple(sorted(lam + (L,), reverse=True))] += w * c * cnt
    return dict(out)

def M_counts(T):
    """Theorem 3.1: dict lambda -> |M(T, lambda)| = TS_lambda(T) * prod d_i!
/ prod T_ij!."""
    num = prod(factorial(sum(r)) for r in T); den = prod(factorial(v) for r i
n T for v in r)
    out = {}

```

```

for lam, c in TS(T).items():
    assert (c * num) % den == 0
    out[lam] = c * num // den
return out

@lru_cache(maxsize=None)
def trail_spectra(T):
    """Corollary 5.3: cycle types realizable in the double coset, computed as
    closed-trail decompositions (no counting)."""
    m = len(T); flat = [v for r in T for v in r]
    if sum(flat) == 0:
        return frozenset([()])
    e = next(k for k, v in enumerate(flat) if v)
    res = set()
    for S in subtables(T, forced=e):
        if not balanced(S) or not connected(S):
            continue
        R = tuple(tuple(T[i][j] - S[i][j] for j in range(m)) for i in range(m))
    L = sum(v for r in S for v in r)
    for lam in trail_spectra(R):
        res.add(tuple(sorted(lam + (L,), reverse=True)))
    return frozenset(res)

# sanity check of the determinant routine against a direct Laplace expansion
def det_Laplace(M):
    if len(M) == 1: return M[0][0]
    return sum((-1) ** j * M[0][j] * det_Laplace([r[:j] + r[j + 1:] for r in M[1:]]) for j in range(len(M)))
import random
random.seed(1)
for _ in range(300):
    k = random.randint(1, 5)
    A = [[random.randint(-6, 6) for _ in range(k)] for _ in range(k)]
    assert det_int(A) == det_Laplace(A)
print("determinant routine checked on 300 random integer matrices")
determinant routine checked on 300 random integer matrices

```

### 3. Lemma 2.1: double cosets of $K = S_\mu$ are exactly the classes of equal table, with Fisher–Yates sizes

The orbits of  $K \times K$  acting by  $\sigma \mapsto k\sigma k'$  are computed by union-find over all permutations, using generators of  $K$  on both sides.

```

def check_double_cosets(n):
    perms = list(itertools.permutations(range(n))); idx = {p: i for i, p in enumerate(perms)}
    total_cosets = mismatches = size_errors = 0
    for mu in partitions(n):
        m = len(mu); b = block_index(mu); start = 0; gens = []
        for d in mu:
            for a in range(start, start + d - 1):
                g = list(range(n)); g[a], g[a + 1] = a + 1, a; gens.append(g)
                start += d
        uf = UnionFind(len(perms))
        for i, p in enumerate(perms):
            for g in gens:
                uf.union(i, idx[tuple(g[p[x]] for x in range(n))])
                uf.union(i, idx[tuple(p[g[x]] for x in range(n))])
        by_orbit = defaultdict(set); by_table = defaultdict(set); orbit_size = Counter()
        for i, p in enumerate(perms):
            T = table_of(p, b, m); r = uf.find(i)
            by_orbit[r].add(T); by_table[T].add(r); orbit_size[r] += 1
            mismatches += sum(len(v) > 1 for v in by_orbit.values()) + sum(len(v) > 1 for v in by_table.values())
            size_errors += sum(orbit_size[r] != fisher_yates(next(iter(Ts))) for r, Ts in by_orbit.items())
            total_cosets += len(by_orbit)
        return total_cosets, mismatches, size_errors

for n in range(1, N_ENUM + 1):
    t0 = time.time(); c, mm, se = check_double_cosets(n)
    print(f"n={n}: double cosets {c:>6}, partition mismatches {mm}, size mismatches {se} [{time.time() - t0:.1f}s]")

n=1: double cosets      1, partition mismatches 0, size mismatches 0 [0.0s]
n=2: double cosets      3, partition mismatches 0, size mismatches 0 [0.0s]
n=3: double cosets      9, partition mismatches 0, size mismatches 0 [0.0s]
n=4: double cosets     37, partition mismatches 0, size mismatches 0 [0.0s]
n=5: double cosets    177, partition mismatches 0, size mismatches 0 [0.0s]
n=6: double cosets   1054, partition mismatches 0, size mismatches 0 [0.1s]
n=7: double cosets   7237, partition mismatches 0, size mismatches 0 [0.6s]

```

#### 4. Theorems 3.1, 4.1 and 5.1, Corollary 5.3, Proposition 5.4: comparison with direct enumeration on every table

For every block type and every permutation, the counts  $|M(T, \lambda)|$  obtained by enumeration are compared with the transition-system recursion. The single-cycle count is compared with the BEST closed form, the

enumerated support with the closed-trail spectra, and the minimum number of cycles with the number of connected components.

```
def components(T):
    m = len(T); parent = list(range(m))
    def f(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]; x = parent[x]
        return x
    for i in range(m):
        for j in range(m):
            if T[i][j]:
                parent[f(i)] = f(j)
    return len({f(i) for i in range(m)})

def single_cycle_formula(T):
    d = [sum(r) for r in T]
    return eulerian_circuits(T) * prod(factorial(x) for x in d) // prod(factorial(v) for r in T for v in r)

for n in range(2, N_ENUM + 1):
    t0 = time.time()
    perms = list(itertools.permutations(range(n))); lam_of = [cycle_type(p) for p in perms]
    n_tables = bad_rec = bad_best = bad_supp = bad_min = 0
    for mu in partitions(n):
        m = len(mu); b = block_index(mu); counts = defaultdict(Counter)
        for p, lam in zip(perms, lam_of):
            counts[table_of(p, b, m)][lam] += 1
        for T, C in counts.items():
            n_tables += 1
            bad_rec += (dict(C) != M_counts(T))
            bad_best += (C.get((n,), 0) != single_cycle_formula(T))
            bad_supp += (set(C) != set(trail_spectra(T)))
            bad_min += (min(len(l) for l in C) != components(T))
    print(f"n={n}: tables {n_tables:>6} | recursion mismatches {bad_rec} | single-cycle formula mismatches {bad_best} | support mismatches {bad_supp} | min-cycles mismatches {bad_min} [{time.time() - t0:.1f}s]")

n=2: tables      3 | recursion mismatches 0 | single-cycle formula mismatches 0 | support mismatches 0 | min-cycles mismatches 0 [0.0s]
n=3: tables      9 | recursion mismatches 0 | single-cycle formula mismatches 0 | support mismatches 0 | min-cycles mismatches 0 [0.0s]
n=4: tables     37 | recursion mismatches 0 | single-cycle formula mismatches
```

```

0 | support mismatches 0 | min-cycles mismatches 0 [0.0s]
n=5: tables 177 | recursion mismatches 0 | single-cycle formula mismatches
0 | support mismatches 0 | min-cycles mismatches 0 [0.0s]
n=6: tables 1054 | recursion mismatches 0 | single-cycle formula mismatches
0 | support mismatches 0 | min-cycles mismatches 0 [0.3s]
n=7: tables 7237 | recursion mismatches 0 | single-cycle formula mismatches
0 | support mismatches 0 | min-cycles mismatches 0 [4.1s]

```

### 5. Example 3.4

```

T = ((1, 1), (1, 1))
print("TS(T) =", TS(T))
print("/M(T, Lambda)| =", M_counts(T), " sum =", sum(M_counts(T).values()),
" Fisher-Yates size =", fisher_yates(T))

TS(T) = {(2, 1, 1): 1, (3, 1): 2, (4,): 1}
|M(T, Lambda)| = {(2, 1, 1): 4, (3, 1): 8, (4,): 4} sum = 16 Fisher-Yates
size = 16

```

### 6. Theorem 4.3 (dominant cycle), Corollaries 4.5 and 4.6, Theorem 4.7 (involutions), Theorem 5.2 (length-rooted recursion)

Each formula is compared with TS, which section 4 has already matched against enumeration on every table for these  $n$ .

```

def balanced_subtables_of_total(T, total):
    m = len(T); flat = [v for r in T for v in r]
    for S in itertools.product(*[range(0, v + 1) for v in flat]):
        if sum(S) != total:
            continue
        Sm = tuple(tuple(S[i * m:(i + 1) * m]) for i in range(m))
        if balanced(Sm):
            yield Sm, S

def minus(T, S):
    m = len(T)
    return tuple(tuple(T[i][j] - S[i][j] for j in range(m)) for i in range(m))

def dominant_cycle_formula(T, Lam):
    """lam = (L) + nu with L > nu_1."""
    nu = Lam[1:]; flat = [v for r in T for v in r]; total = 0
    for R, Rflat in balanced_subtables_of_total(T, sum(nu)):
        t_nu = TS(R).get(nu, 0) if nu else 1
        if t_nu:
            total += prod(comb(t, r) for t, r in zip(flat, Rflat)) * t_nu * e
    ulerian_circuits(minus(T, R))

```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

[github.com/QuHarmonics/The-Nexus-Harmonic-Reality](https://github.com/QuHarmonics/The-Nexus-Harmonic-Reality) [info@quharmonics.com](mailto:info@quharmonics.com)

```

    return total

def hook_formula(T, k):
    m = len(T); total = 0
    for a in itertools.product(*[range(T[v][v] + 1) for v in range(m)]):
        if sum(a) == k:
            R = tuple(tuple(T[i][j] - (a[i] if i == j else 0) for j in range(
m)) for i in range(m))
            total += prod(comb(T[v][v], a[v]) for v in range(m)) * eulerian_c
ircuits(R)
    return total

def two_one_formula(T, j, k):
    """lambda = (L, 2^j, 1^k), L >= 3."""
    m = len(T); pairs = [(i, l) for i in range(m) for l in range(i + 1, m)];
    total = 0
    for a in itertools.product(*[range(T[v][v] + 1) for v in range(m)]):
        if sum(a) != k:
            continue
        for c in itertools.product(*[range((T[v][v] - a[v]) // 2 + 1) for v i
n range(m)]):
            if sum(c) > j:
                continue
            for bb in itertools.product(*[range(min(T[i][l], T[l][i]) + 1) fo
r i, l in pairs]):
                if sum(c) + sum(bb) != j:
                    continue
                R = [list(r) for r in T]; w = 1
                for v in range(m):
                    w *= comb(T[v][v], a[v]) * comb(T[v][v] - a[v], 2 * c[v])
                * factorial(2 * c[v]) // (2 ** c[v] * factorial(c[v]))
                R[v][v] -= a[v] + 2 * c[v]
                for (i, l), x in zip(pairs, bb):
                    w *= comb(T[i][l], x) * comb(T[l][i], x) * factorial(x);
                R[i][l] -= x; R[l][i] -= x
                if w:
                    total += w * eulerian_circuits(R)
    return total

def involution_formula(T, k):
    m = len(T)
    if T != transpose(T):
        return 0
    poly = {0: 1}

```

```

for v in range(m):
    new = Counter()
    for a in range(T[v][v] + 1):
        r = T[v][v] - a
        if r % 2:
            continue
        w = comb(T[v][v], a) * prod(range(r - 1, 0, -2)) # (r-1)!! with
(-1)!! = 1
        for e, c in poly.items():
            new[e + a] += c * w
        poly = new
    return prod(factorial(T[i][L]) for i in range(m) for L in range(i + 1, m)
) * poly.get(k, 0)

def length_rooted_formula(T, lam):
    L = lam[0]; rest = lam[1:]; mult = lam.count(L); flat = [v for r in T for
v in r]; total = 0
    for S, Sflat in balanced_subtables_of_total(T, L):
        c = eulerian_circuits(S)
        if c == 0:
            continue
        R = minus(T, S)
        t_rest = TS(R).get(rest, 0) if rest else (1 if sum(v for r in R for v
in r) == 0 else 0)
        total += prod(comb(t, s) for t, s in zip(flat, Sflat)) * c * t_rest
    assert total % mult == 0
    return total // mult

t0 = time.time()
stats = Counter()
for n in range(2, 8):
    for mu in partitions(n):
        for T in tables(mu):
            ts = TS(T)
            for lam in ts:
                if len(lam) == 1 or lam[0] > lam[1]:
                    stats["dominant cases"] += 1; stats["dominant mismatches"]
] += dominant_cycle_formula(T, lam) != ts[lam]
                    stats["length-rooted cases"] += 1; stats["length-rooted misma
tches"] += length_rooted_formula(T, lam) != ts[lam]
                for k in range(0, n - 1):
                    stats["hook cases"] += 1; stats["hook mismatches"] += hook_fo
rmula(T, k) != ts.get((n - k,) + (1,) * k, 0)
                for j in range(0, (n - 3) // 2 + 1):

```

```

    for k in range(0, n - 2 * j - 2):
        lam = (n - 2 * j - k,) + (2,) * j + (1,) * k
        stats["(L,2^j,1^k) cases"] += 1; stats["(L,2^j,1^k) mismatches"] += two_one_formula(T, j, k) != ts.get(lam, 0)
    for k in range(n % 2, n + 1, 2):
        lam = (2,) * ((n - k) // 2) + (1,) * k
        stats["involution cases"] += 1; stats["involution mismatches"] += involution_formula(T, k) != ts.get(lam, 0)
    for key in ["dominant", "hook", "(L,2^j,1^k)", "involution", "length-rooted"]:
        print(f"{key:>14}: {stats[key + ' cases']:>6} cases, {stats[key + ' mismatches']} mismatches (all tables, 2 <= n <= 7)")
    print(f"[{time.time() - t0:.1f}s]")

    dominant: 12038 cases, 0 mismatches (all tables, 2 <= n <= 7)
    hook: 49532 cases, 0 mismatches (all tables, 2 <= n <= 7)
    (L,2^j,1^k): 72248 cases, 0 mismatches (all tables, 2 <= n <= 7)
    involution: 33830 cases, 0 mismatches (all tables, 2 <= n <= 7)
    length-rooted: 13706 cases, 0 mismatches (all tables, 2 <= n <= 7)
[4.5s]

```

## 7. Theorem 6.2: census over all pairs $(P, \sigma)$ with $K \neq 1$ (Table 1)

Pairs are weighted by the number of set partitions of each block type.

```

for n in (5, 6, 7):
    t0 = time.time()
    perms = list(itertools.permutations(range(n))); lam_of = [cycle_type(p) for p in perms]; class_size = Counter(lam_of)
    pairs = strict = ties = ties_coset_smaller = 0; log_gain = 0.0
    for mu in partitions(n):
        if all(d == 1 for d in mu):
            continue
        weight = n_set_partitions_of_type(mu); m = len(mu); b = block_index(mu)
        tabs = [table_of(p, b, m) for p in perms]; counts = defaultdict(Counter)
        for T, lam in zip(tabs, lam_of):
            counts[T][lam] += 1
        for T, lam in zip(tabs, lam_of):
            M = counts[T][lam]; D = fisher_yates(T); Cl = class_size[lam]; smaller = min(D, Cl)
            pairs += weight
            if M < smaller:
                strict += weight
            else:

```

```

    ties += weight; ties_coset_smaller += weight if D <= Cl else
0
    log_gain += weight * (math.log2(smaller) - math.log2(M))
    print(f"n={n}: pairs {pairs:>9,} | strictly smaller {strict:>9,} | equal
to minimum {ties:>6,} | "
        f"of which double coset is the smaller set {ties_coset_smaller} | m
ean log2(min/|M|) {log_gain / pairs:.3f} [{time.time() - t0:.1f}s]")
n=5: pairs      6,120 | strictly smaller      5,730 | equal to minimum      390 |
of which double coset is the smaller set 0 | mean log2(min/|M|) 1.193 [0.0
s]
n=6: pairs    145,440 | strictly smaller    142,929 | equal to minimum    2,511 |
of which double coset is the smaller set 0 | mean log2(min/|M|) 1.685 [0.0
s]
n=7: pairs 4,415,040 | strictly smaller 4,396,147 | equal to minimum 18,893 |
of which double coset is the smaller set 0 | mean log2(min/|M|) 2.221 [0.3
s]

```

## 8. Section 7: orbit counts, profile counts and Theorem 7.1 (Table 2)

```

def set_partitions(n):
    def rec(i, blocks):
        if i == n:
            yield [tuple(bl) for bl in blocks]; return
        for bl in blocks:
            bl.append(i); yield from rec(i + 1, blocks); bl.pop()
            blocks.append([i]); yield from rec(i + 1, blocks); blocks.pop()
        yield from rec(0, [])

def burnside_orbits(n):
    """|X/S_n| and |X/(S_n x C_2)| for X = {(P, sigma)}, by Burnside's lemma.
    """
    perms = list(itertools.permutations(range(n)))
    parts = [frozenset(frozenset(bl) for bl in P) for P in set_partitions(n)]
    class_size = Counter(cycle_type(p) for p in perms); reps = {}
    for p in perms:
        reps.setdefault(cycle_type(p), p)
    s1 = s2 = 0
    for lam, g in reps.items():
        fixed_partitions = sum(1 for P in parts if frozenset(frozenset(g[x] f
or x in bl) for bl in P) == P)
        commuting = twisted = 0
        for s in perms:
            if all(g[s[x]] == s[g[x]] for x in range(n)):
                commuting += 1
            if all(s[g[s[x]]] == g[x] for x in range(n)): # g s^{-1} g^{-1}

```

```

= s
        twisted += 1
        s1 += class_size[lam] * fixed_partitions * commuting
        s2 += class_size[lam] * fixed_partitions * (commuting + twisted)
    assert s1 % factorial(n) == 0 and s2 % (2 * factorial(n)) == 0
    return s1 // factorial(n), s2 // (2 * factorial(n))

def table_classes(mu, with_transpose):
    tabs = tables(mu); index = {t: i for i, t in enumerate(tabs)}; uf = UnionFind(len(tabs)); m = len(mu)
    for t, i in index.items():
        for s in range(m - 1):
            if mu[s] == mu[s + 1]:
                p = list(range(m)); p[s], p[s + 1] = s + 1, s
                uf.union(i, index[relabel(t, p)])
        if with_transpose:
            uf.union(i, index[transpose(t)])
    reps = {}
    for t, i in index.items():
        reps.setdefault(uf.find(i), t)
    return len(tabs), list(reps.values())

def profile_counts(n):
    """labeled tables, table classes (without/with transpose), profiles (with
    out/with transpose), distinct |M| values"""
    n_tab = c1 = c2 = p1 = p2 = 0; values = set()
    for mu in partitions(n):
        nt, reps1 = table_classes(mu, False); _, reps2 = table_classes(mu, True)
        n_tab += nt; c1 += len(reps1); c2 += len(reps2)
        p1 += sum(len(trail_spectra(t)) for t in reps1); p2 += sum(len(trail_
        spectra(t)) for t in reps2)
        for t in reps2:
            values |= set(M_counts(t).values())
    return n_tab, c1, c2, p1, p2, len(values)

def direct_orbits(n):
    """Direct union-find over orbit representatives: conjugation orbits, conj
    ugation+inversion orbits,
    classes of the relation generated by conjugation and tau in M(P, sigma) (
    without/with inversion),
    and the number of conjugation+inversion orbits on which the profile witho
    ut transpose is not constant."""
    perms = list(itertools.permutations(range(n))); idx = {p: i for i, p in e

```

```

numerate(perms)}
    inv = [idx[inverse(p)] for p in perms]; lam_of = [cycle_type(p) for p in
perms]
    o1 = o2 = r1 = r2 = split = 0
    for mu in partitions(n):
        m = len(mu); b = block_index(mu); blocks = []; start = 0
        for d in mu:
            blocks.append(list(range(start, start + d))); start += d
        gens = []
        for bl in blocks:
            for x, y in zip(bl, bl[1:]):
                g = list(range(n)); g[x], g[y] = y, x; gens.append(g)
        for s in range(m - 1):
            if mu[s] == mu[s + 1]:
                g = list(range(n))
                for x, y in zip(blocks[s], blocks[s + 1]):
                    g[x], g[y] = y, x
                gens.append(g)
        ufs = [UnionFind(len(perms)) for _ in range(4)] # conj, conj+inv, c
onj+rel, conj+rel+inv
        cell = defaultdict(list); tab = []
        for i, p in enumerate(perms):
            T = table_of(p, b, m); tab.append(T); cell[(T, lam_of[i])].append
(i)
            for g in gens: # g is an involut
ion, so  $g p g^{-1} = g p g$ 
                j = idx[tuple(g[p[g[x]]] for x in range(n))]
                for uf in ufs:
                    uf.union(i, j)
                ufs[1].union(i, inv[i]); ufs[3].union(i, inv[i])
        for members in cell.values():
            for j in members[1:]:
                ufs[2].union(members[0], j); ufs[3].union(members[0], j)
        o1 += ufs[0].count(); o2 += ufs[1].count(); r1 += ufs[2].count(); r2
+= ufs[3].count()
        # class of each labeled table under size-preserving relabeling (union
-find on tables)
        tabs = tables(mu); tindex = {t: k for k, t in enumerate(tabs)}; tuf =
UnionFind(len(tabs))
        for t, k in tindex.items():
            for s in range(m - 1):
                if mu[s] == mu[s + 1]:
                    p = list(range(m)); p[s], p[s + 1] = s + 1, s
                    tuf.union(k, tindex[relabel(t, p)])

```

```

    profiles_in_orbit = defaultdict(set)
    for i, T in enumerate(tab):
        profiles_in_orbit[ufs[1].find(i)].add((tuf.find(tindex[T]), Lam_o
f[i]))
        split += sum(len(v) > 1 for v in profiles_in_orbit.values())
    return o1, o2, r1, r2, split

print(" n | tables | classes ~ / ~t | |X/S_n| | |X/(S_n x C2)| | profiles | p
rofiles, transpose-merged | values of F")
for n in range(1, N_TABLE + 1):
    t0 = time.time()
    b1, b2 = burnside_orbits(n)
    nt, c1, c2, p1, p2, nv = profile_counts(n)
    line = f"{n:>2} | {nt:>7,} | {c1:>5} / {c2:<5} | {b1:>7,} | {b2:>7,} | {p
1:>7,} | {p2:>7,} | {nv:>4}"
    if n <= 7:
        o1, o2, r1, r2, split = direct_orbits(n)
        agree = (o1, o2, r1, r2) == (b1, b2, p1, p2)
        line += f"    direct union-find agrees: {agree}; inversion-orbits spli
t by untransposed profile: {split}"
        print(line + f"    [{time.time() - t0:.1f}s]")

    n | tables | classes ~ / ~t | |X/S_n| | |X/(S_n x C2)| | profiles | profiles
, transpose-merged | values of F
1 |      1 |      1 / 1      |      1 |      1 |      1 |      1 |      1
direct union-find agrees: True; inversion-orbits split by untransposed profi
le: 0    [0.0s]
2 |      3 |      3 / 3      |      4 |      4 |      4 |      4 |      1
direct union-find agrees: True; inversion-orbits split by untransposed profi
le: 0    [0.0s]
3 |      9 |      6 / 6      |     10 |     10 |     10 |     10 |      3
direct union-find agrees: True; inversion-orbits split by untransposed profi
le: 0    [0.0s]
4 |     37 |     16 / 16      |     35 |     35 |     35 |     35 |      6
direct union-find agrees: True; inversion-orbits split by untransposed profi
le: 0    [0.0s]
5 |    177 |     34 / 34      |    102 |    101 |     97 |     97 |     13
direct union-find agrees: True; inversion-orbits split by untransposed profi
le: 0    [0.0s]
6 |   1,054 |     95 / 92      |    395 |    372 |    359 |    344 |     30
direct union-find agrees: True; inversion-orbits split by untransposed profi
le: 18   [0.2s]
7 |   7,237 |    236 / 220      |   1,480 |   1,297 |   1,181 |   1,086 |     60
direct union-find agrees: True; inversion-orbits split by untransposed profi
le: 122  [1.9s]

```

## 9. Section 8: how much the table determines (Table 3)

The measure is uniform on  $X$ . The entropies are exact finite sums over all labeled tables, weighted by the number of set partitions of each type.

```
def entropy(ps):
    return -sum(p * math.log2(p) for p in ps if p > 0)

for n in range(4, N_ENUM + 1):
    t0 = time.time(); N = factorial(n)
    B = sum(n_set_partitions_of_type(mu) for mu in partitions(n))
    H_Lam_T = H_F_T = P_Lam = P_F = 0.0; Lam_marg = Counter(); F_marg = Counter()
    for mu in partitions(n):
        w = n_set_partitions_of_type(mu) / B
        for T in tables(mu):
            MV = M_counts(T); D = sum(MV.values()); pT = D / N
            mass_by_value = Counter()
            for lam, v in MV.items():
                mass_by_value[v] += v; Lam_marg[lam] += w * v / N; F_marg[v]
            += w * v / N
            H_Lam_T += w * pT * entropy([v / D for v in MV.values()])
            H_F_T += w * pT * entropy([c / D for c in mass_by_value.values()])
        )
        P_Lam += w * pT * (len(MV) == 1); P_F += w * pT * (len(mass_by_value) == 1)
    H_Lam = entropy(Lam_marg.values()); H_F = entropy(F_marg.values())
    print(f"n={n}: H(Lambda)={H_Lam:.3f} H(Lambda|T)={H_Lam_T:.3f} I(Lambda;T)={H_Lam - H_Lam_T:.3f} "
          f"H(F)={H_F:.3f} H(F|T)={H_F_T:.3f} P_Lambda={P_Lam:.4f} P_F={P_F:.4f} [{time.time() - t0:.1f}s]")

n=4: H(Lambda)=2.094 H(Lambda|T)=1.304 I(Lambda;T)=0.790 H(F)=2.305 H(F|T)=0.554 P_Lambda=0.0667 P_F=0.5000 [0.0s]
n=5: H(Lambda)=2.557 H(Lambda|T)=1.718 I(Lambda;T)=0.839 H(F)=3.051 H(F|T)=0.828 P_Lambda=0.0192 P_F=0.3750 [0.0s]
n=6: H(Lambda)=3.035 H(Lambda|T)=2.121 I(Lambda;T)=0.914 H(F)=3.747 H(F|T)=1.136 P_Lambda=0.0049 P_F=0.2332 [0.0s]
n=7: H(Lambda)=3.436 H(Lambda|T)=2.517 I(Lambda;T)=0.919 H(F)=4.271 H(F|T)=1.465 P_Lambda=0.0011 P_F=0.1349 [0.1s]
```

## 10. Section 8: level sets of $F$ (Table 4) and the values at $n = 7$ (Appendix C)

```
for n in range(6, N_TABLE + 1):
    t0 = time.time(); level = defaultdict(list)
    for mu in partitions(n):
        for T in table_classes(mu, True)[1]:
```

Copyright Dean A. Kulik – Orcid ID # 0009-0003-3128-8828

Creative Commons Attribution-NonCommercial 4.0 International License (CC BY-NC 4.0)

github.com/QuHarmonics/The-Nexus-Harmonic-Reality info@quharmonics.com

```

    for lam, v in M_counts(T).items():
        Level[v].append((mu, lam))
    profiles = sum(len(v) for v in Level.values())
    across_types = sum(len({mu for mu, _ in cells}) > 1 for cells in Level.values())
    across_cycles = sum(len({len(lam) for _, lam in cells}) > 1 for cells in Level.values())
    top = max(Level.items(), key=lambda kv: len(kv[1]))
    print(f"n={n}: profiles {profiles} | values {len(Level)} | values shared by >=2 block types {across_types} | "
          f"by >=2 cycle counts {across_cycles} | Largest Level set F={top[0]} with {len(top[1]):,} profiles [{time.time() - t0:.1f}s]")
    if n == 7:
        print("    values at n=7:", sorted(Level))

n=6: profiles 344 | values 30 | values shared by >=2 block types 21 | by >=2 cycle counts 22 | Largest Level set F=2 with 53 profiles [0.0s]
n=7: profiles 1,086 | values 60 | values shared by >=2 block types 40 | by >=2 cycle counts 43 | Largest Level set F=4 with 131 profiles [0.3s]
    values at n=7: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 15, 16, 18, 20, 21, 24, 25, 28, 30, 32, 36, 40, 45, 48, 50, 54, 60, 64, 70, 72, 84, 90, 96, 100, 105, 108, 120, 140, 144, 160, 168, 210, 216, 240, 260, 280, 288, 300, 324, 360, 420, 432, 480, 504, 540, 630, 720, 840]

```

## 11. Appendices A and B

```

print("Appendix A: table classes at n = 4 (without transpose)")
for mu in partitions(4):
    for T in sorted(table_classes(mu, False)[1], reverse=True):
        print(f"    mu={mu} T={T} |K sigma K|={fisher_yates(T)} |M|:", dict(sorted(M_counts(T).items(), reverse=True)))
print()
print("Appendix B: n = 7 by block type: tables, classes ~, profiles, classes ~t, profiles (transpose-merged)")
tot = [0] * 5
for mu in partitions(7):
    nt, r1 = table_classes(mu, False); _, r2 = table_classes(mu, True)
    row = [nt, len(r1), sum(len(trail_spectra(t)) for t in r1), len(r2), sum(len(trail_spectra(t)) for t in r2)]
    tot = [a + b for a, b in zip(tot, row)]
    print(f"    {str(mu):<22}", row)
print("    totals:", tot)

```

### Appendix A: table classes at n = 4 (without transpose)

```

mu=(4,) T=((4,)) |K sigma K|=24 |M|: {(4,): 6, (3, 1): 8, (2, 2): 3, (2, 1, 1): 6, (1, 1, 1, 1): 1}
mu=(3, 1) T=((3, 0), (0, 1)) |K sigma K|=6 |M|: {(3, 1): 2, (2, 1, 1): 3

```

```
, (1, 1, 1, 1): 1}
mu=(3, 1) T=((2, 1), (1, 0)) |K sigma K|=18 |M|: {(4,): 6, (3, 1): 6, (2,
2): 3, (2, 1, 1): 3}
mu=(2, 2) T=((2, 0), (0, 2)) |K sigma K|=4 |M|: {(2, 2): 1, (2, 1, 1): 2
, (1, 1, 1, 1): 1}
mu=(2, 2) T=((1, 1), (1, 1)) |K sigma K|=16 |M|: {(4,): 4, (3, 1): 8, (2
, 1, 1): 4}
mu=(2, 2) T=((0, 2), (2, 0)) |K sigma K|=4 |M|: {(4,): 2, (2, 2): 2}
mu=(2, 1, 1) T=((2, 0, 0), (0, 1, 0), (0, 0, 1)) |K sigma K|=2 |M|: {(2,
1, 1): 1, (1, 1, 1, 1): 1}
mu=(2, 1, 1) T=((2, 0, 0), (0, 0, 1), (0, 1, 0)) |K sigma K|=2 |M|: {(2,
2): 1, (2, 1, 1): 1}
mu=(2, 1, 1) T=((1, 1, 0), (1, 0, 0), (0, 0, 1)) |K sigma K|=4 |M|: {(3,
1): 2, (2, 1, 1): 2}
mu=(2, 1, 1) T=((1, 1, 0), (0, 0, 1), (1, 0, 0)) |K sigma K|=4 |M|: {(4,
): 2, (3, 1): 2}
mu=(2, 1, 1) T=((0, 1, 1), (1, 0, 0), (1, 0, 0)) |K sigma K|=4 |M|: {(4,
): 2, (2, 2): 2}
mu=(1, 1, 1, 1) T=((1, 0, 0, 0), (0, 1, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1))
|K sigma K|=1 |M|: {(1, 1, 1, 1): 1}
mu=(1, 1, 1, 1) T=((1, 0, 0, 0), (0, 1, 0, 0), (0, 0, 0, 1), (0, 0, 1, 0))
|K sigma K|=1 |M|: {(2, 1, 1): 1}
mu=(1, 1, 1, 1) T=((1, 0, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1), (0, 1, 0, 0))
|K sigma K|=1 |M|: {(3, 1): 1}
mu=(1, 1, 1, 1) T=((0, 1, 0, 0), (1, 0, 0, 0), (0, 0, 0, 1), (0, 0, 1, 0))
|K sigma K|=1 |M|: {(2, 2): 1}
mu=(1, 1, 1, 1) T=((0, 1, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1), (1, 0, 0, 0))
|K sigma K|=1 |M|: {(4,): 1}
```

Appendix B:  $n = 7$  by block type: tables, classes ~, profiles, classes ~t, pro  
files (transpose-merged)

|                 |                         |
|-----------------|-------------------------|
| (7,)            | [1, 1, 15, 1, 15]       |
| (6, 1)          | [2, 2, 25, 2, 25]       |
| (5, 2)          | [3, 3, 37, 3, 37]       |
| (5, 1, 1)       | [7, 5, 47, 5, 47]       |
| (4, 3)          | [4, 4, 44, 4, 44]       |
| (4, 2, 1)       | [12, 12, 112, 10, 93]   |
| (4, 1, 1, 1)    | [34, 10, 65, 10, 65]    |
| (3, 3, 1)       | [16, 10, 75, 10, 75]    |
| (3, 2, 2)       | [25, 15, 114, 14, 106]  |
| (3, 2, 1, 1)    | [67, 39, 228, 31, 183]  |
| (3, 1, 1, 1, 1) | [208, 19, 72, 19, 72]   |
| (2, 2, 2, 1)    | [120, 26, 125, 24, 114] |
| (2, 2, 1, 1, 1) | [378, 47, 151, 44, 139] |

(2, 1, 1, 1, 1, 1) [1320, 28, 56, 28, 56]  
(1, 1, 1, 1, 1, 1, 1) [5040, 15, 15, 15, 15]  
totals: [7237, 236, 1181, 220, 1086]